```
  end
end

obj = Special.new("Hello", "there", "World")
puts "Before: obj = #{obj}"
data = Marshal.dump(obj)

obj = Marshal.load(data)
puts "After: obj = #{obj}"
```

produces:

```
Before: obj = Hello there World
After: obj = Hello unknown World
```

Using YAML and JSON for Marshaling

The Marshal module is built into the interpreter and uses a binary format to store objects externally. Although fast, this binary format has a couple of disadvantages: if the interpreter changes significantly, the marshal binary format may also change and old dumped files may no longer be loadable. Also, using Marshal assumes that the code on the other end loading the object is also a Ruby program. You might be sending an object across the web on a remote call to another server that might not be in Ruby, and in that case, you'll want a more general conversion format.

An alternative is to use a less fussy external format, preferably one using text rather than binary files. One option, supplied as a standard library, is YAML.[1] YAML stands for YAML Ain't Markup Language, but that hardly seems important. Another common option in the standard library is JavaScript Object Notation, or JSON.[2] In Ruby, the YAML module is an alias for Psych, which is the name of the Ruby YAML parser.

We can adapt our previous marshal example to use YAML. Rather than implement specific loading and dumping methods to control the marshal process, we simply define the method encode_with, which explicitly sets the values to be saved into its parameter:

ospace/yaml_load.rb

```
require "yaml"

class Special
  def initialize(valuable, volatile, precious)
    @valuable = valuable
    @volatile = volatile
    @precious = precious
  end

  def encode_with(properties)
    properties["precious"] = @precious
    properties["valuable"] = @valuable
  end

  def to_s
    "#{@valuable} #{@volatile} #{@precious}"
  end
end

obj = Special.new("Hello", "there", "World")
```

1. http://www.yaml.org
2. https://www.json.org/json-en.html